SECTION A — SCHEMA ANALYSIS AND NORMALIZATION

Given Schema :

```
Bookings (
  booking_id,
  user_name,
  user_email,
  movie_name,
  theatre_name,
  theatre_location,
  show_time,
  seat_numbers,
  total_price,
  payment_status
)
```

A1. Issues in the Given Schema :

The given table tries to store *everything in one place*, which creates multiple problems:

1. Unnecessary Repetition of Data :

The same information is stored multiple times across different rows. For example, if a user books multiple tickets, their name and email are repeated in every booking. Similarly, movie and theatre details are duplicated for every record.

Why this is a problem:
This increases storage usage and makes the database harder to maintain. Even a small change (like correcting an email) has to be done in multiple places.

2. Difficulty in Updating Information :

Because the same data appears in many rows, updating it becomes risky. For instance, if a theatre changes its location, all rows containing that theatre must be updated.

Why this is a problem:
If even one row is missed, the database will contain inconsistent information, which reduces reliability.

3. Inability to Store Independent Data :

The current design does not allow storing certain data unless other related data is present. For example, you cannot add a new movie or theatre unless a booking has already been made for it.

Why this is a problem:
It limits flexibility and prevents the database from storing complete information about the system.

4. Loss of Important Data :

If a booking record is deleted, all related information about that movie or theatre might also be lost if it does not exist elsewhere.

Why this is a problem:
Important details disappear unintentionally, which should not happen in a well-designed database.

A2. Normalization of the Schema :

To improve the original design, the large “Bookings” table is broken into smaller, meaningful tables. Each table now represents a single real-world entity (like User, Movie, Theatre), and relationships are created using keys. This removes redundancy and ensures the database follows Third Normal Form (3NF).

1. Users Table

This table stores user-related information separately so that it is not repeated for every booking.

Attributes:

- user_id
- user_name
- user_email

Primary Key:

- user_id

2. Movie Table

This table keeps information about movies independently.

Attributes:

- movie_id
- movie_name

Primary Key:

- movie_id

3.Theaters Table

This table stores theatre details.

Attributes:

- theatre_id
- theatre_name
- theatre_location

Primary Key:

- theatre_id

4. Show Table (optional)

This table connects movies and theatres along with show timing.

Attributes:

- show_id
- movie_id
- theatre_id
- show_time

Primary Key:

- show_id

Foreign Keys:

- movie_id → Movies(movie_id)
- theatre_id → Theatres(theatre_id)

5.Booking Table

This table records each booking made by users.

Attributes:

- booking_id
- user_id
- show_id
- total_price
- payment_status

Primary Key:

- booking_id

Foreign Keys:

- user_id → Users(user_id)
- show_id → Shows(show_id)

6. Seats Table

This table is created to handle multiple seats in a proper way.

Attributes:

- seat_id
- booking_id
- seat_number

Primary Key:

- seat_id

Foreign Key:

- booking_id → Bookings(booking_id)

A3. Assumptions Made During Schema Redesign

1.unique identity for each User

Every user in the system is assumed to have a distinct `user_id`. In addition, email addresses are treated as unique to avoid duplicate accounts and to maintain reliable identification.

2. Users can book multiple times

A single user is allowed to make multiple bookings over time, whether for different movies or different show timings. This naturally creates a one-to-many relationship between users and bookings.

3. Movies and theatres are independent entities

Movies and theatres are stored separately and are not dependent on bookings. This means they can exist in the system even if no tickets have been booked for them yet.

4. A theatre can host multiple shows

It is assumed that one theatre is capable of screening different movies at different time slots. This is why show details are maintained separately instead of being directly stored in bookings.

5. A show is uniquely defined by its components

Each show is identified as a combination of:

- a specific movie
- a specific theatre
- a specific time

This ensures that every show entry represents a clear and unique event.

6. One booking is linked to one show

Each booking is created for a particular show only. If a user wants tickets for another show, it is recorded as a separate booking entry.

7. Multiple seats per booking are handled separately

Instead of storing all seat numbers in one field, each seat is recorded individually and linked to the booking. This keeps the data clean and follows normalization rules.

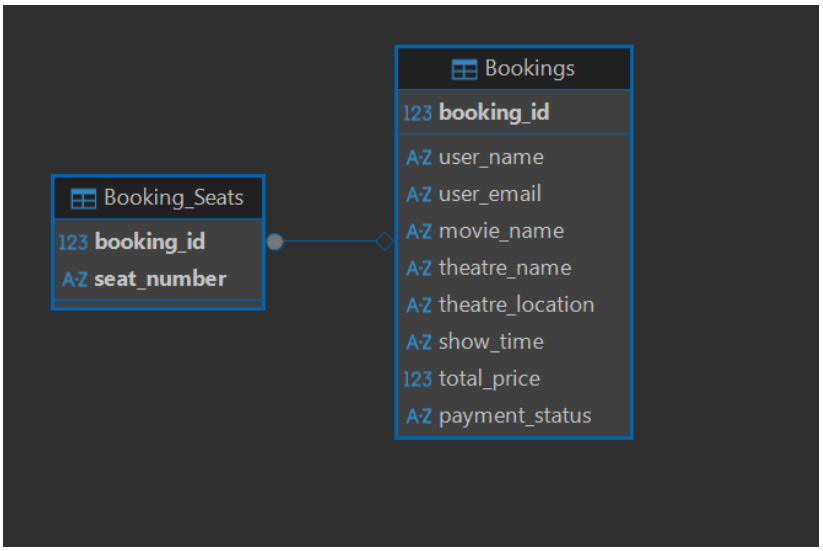
8. Seat duplication is not allowed within a show

For any given show, a seat (for example, A1) can be booked only once. This prevents conflicts and ensures proper seat allocation.

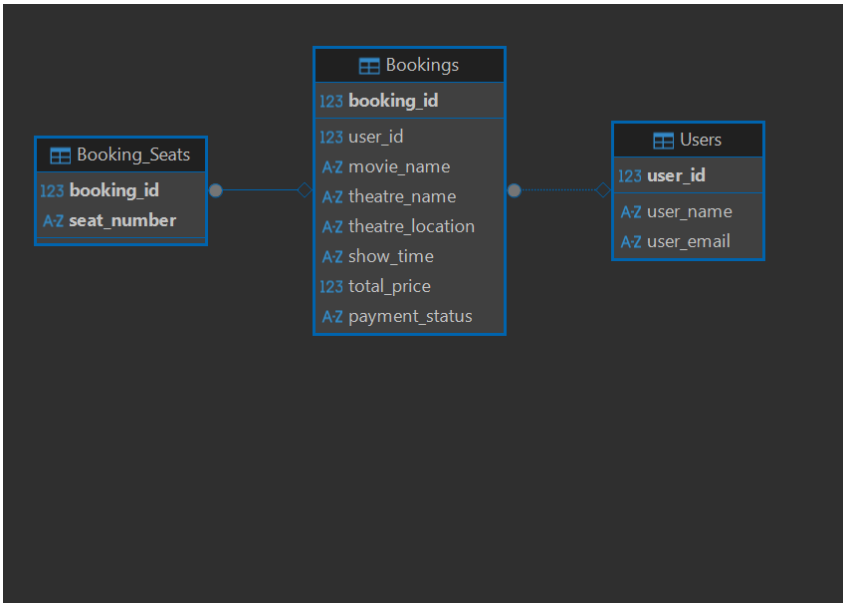
9. Payment status applies to the whole booking
Payment-related information (such as paid, pending, or failed) is associated with the entire booking rather than individual seats.

10. Total price is stored at booking level
The total price represents the combined cost of all seats booked in a single transaction and is stored within the booking record.

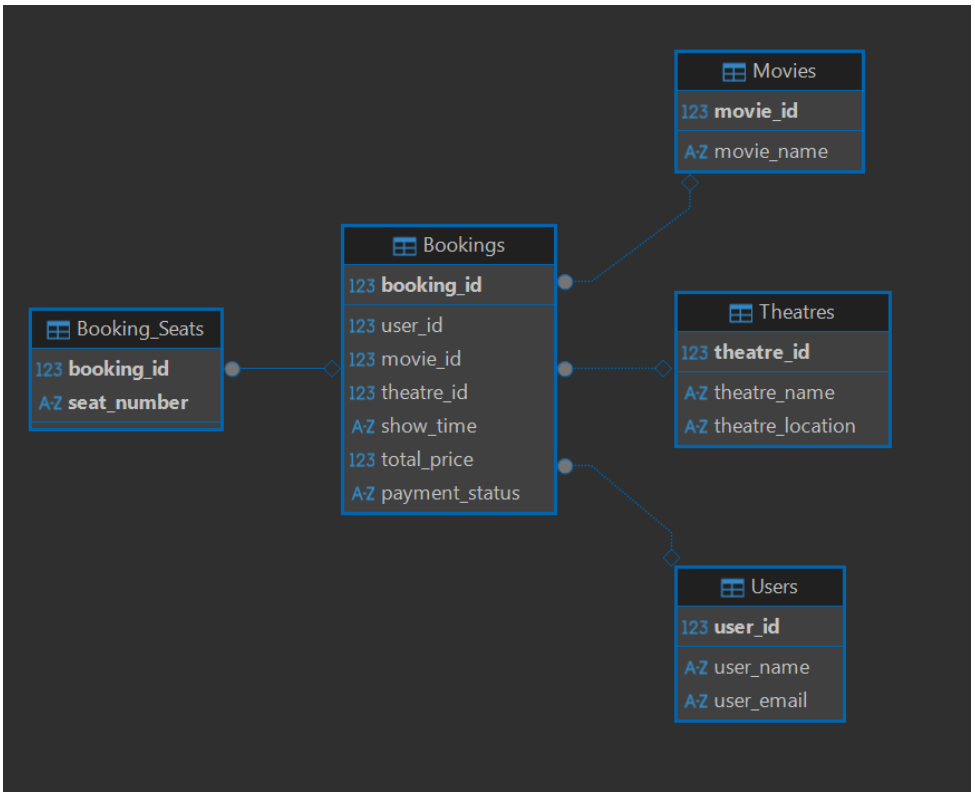
1 NF :



2NF:


Bookings
123 booking_id
123 user_id
A·Z movie_name
A·Z theatre_name
A·Z theatre_location
A·Z show_time
123 total_price
A·Z payment_status
Booking_Seats
123 booking_id
A·Z seat_number
Users
123 user_id
A·Z user_name
A·Z user_email

3NF:


Movies
123 movie_id
A·Z movie_name
Bookings
123 booking_id
123 user_id
123 movie_id
123 theatre_id
A·Z show_time
123 total_price
A·Z payment_status
Booking_Seats
123 booking_id
A·Z seat_number
Theatres
123 theatre_id
A·Z theatre_name
A·Z theatre_location
Users
123 user_id
A·Z user_name
A·Z user_email

SECTION B — DATABASE DESIGN FROM REQUIREMENTS

B1. Requirement Clarification

Before designing the database, a few important points need to be clarified to avoid confusion later and to ensure the system behaves correctly:

1. Structure of theatres and screens

Should each theatre belong to only one city, and can a theatre have multiple screens with different seating layouts?

2. Seat configuration and pricing

Are seat types (like regular, premium) fixed for a screen, or can pricing change depending on the show timing or demand?

3. Booking flexibility

Can users cancel or modify their bookings, and how should that impact seat availability and payment status?

4. Payment handling

Do we only need to track payment status (paid, pending, failed), or should detailed payment information also be stored?

5. Movie attributes

Can a single movie belong to multiple genres and be available in multiple languages at the same time?

6. Multi-user booking scenario

When multiple users are part of one booking, how exactly should seats be assigned to each individual user?

1. Users Table

Stores information about all registered users.

Attributes:

- user_id
- name
- email

Primary Key:

- user_id

2. Cities Table

Represents different locations where the platform operates.

Attributes:

- city_id
- city_name

Primary Key:

- city_id

3. Theatres Table

Contains details of theatres available in different cities.

Attributes:

- theatre_id
- theatre_name
- city_id

Primary Key:

- theatre_id

Foreign Key:

- city_id → Cities(city_id)

4. Screens Table

Each theatre can have multiple screens.

Attributes:

- screen_id
- theatre_id
- screen_name

Primary Key:

- screen_id

Foreign Key:

- theatre_id → Theatres(theatre_id)

5. Movies Table

Stores basic movie information.

Attributes:

- movie_id
- movie_name

Primary Key:

- movie_id

6. Genres Table

Stores different movie genres.

Attributes:

- genre_id
- genre_name

Primary Key:

- genre_id

7. Movie_Genre Table

Handles multiple genres for a movie.

Attributes:

- movie_id
- genre_id

Primary Key:

- (movie_id, genre_id)

Foreign Keys:

- movie_id → Movies(movie_id)
- genre_id → Genres(genre_id)

8. Languages Table

Stores supported languages.

Attributes:

- language_id
- language_name

Primary Key:

- language_id

9. Movie_Language Table

Maps movies to multiple languages.

Attributes:

- movie_id
- language_id

Primary Key:

- (movie_id, language_id)

Foreign Keys:

- movie_id → Movies(movie_id)
- language_id → Languages(language_id)

10. Seats Table

Defines seat details for each screen.

Attributes:

- seat_id
- screen_id
- seat_number
- seat_type

Primary Key:

- seat_id

Foreign Key:

- screen_id → Screens(screen_id)

11. Shows Table

Represents a movie being played on a specific screen at a given time.

Attributes:

- show_id
- movie_id
- screen_id
- show_time

Primary Key:

- show_id

Foreign Keys:

- movie_id → Movies(movie_id)
- screen_id → Screens(screen_id)

12. Show_Seats Table

Tracks seat availability and pricing for each show.

Attributes:

- show_seat_id
- show_id
- seat_id
- price
- is_available

Primary Key:

- show_seat_id

Foreign Keys:

- show_id → Shows(show_id)
- seat_id → Seats(seat_id)

13. Bookings Table

Stores booking-level information.

Attributes:

- booking_id
- booking_time
- payment_status

Primary Key:

- booking_id

14. Booking_Seats Table

Links bookings with the seats selected.

Attributes:

- booking_id
- show_seat_id

Primary Key:

- (booking_id, show_seat_id)

Foreign Keys:

- booking_id → Bookings(booking_id)
- show_seat_id → Show_Seats(show_seat_id)

B3. Normalization

First Normal Form (1NF) :

All tables are designed so that each column holds a single, indivisible value. For example, seat details are not stored as a list in one field; instead, each seat is recorded separately. This ensures the data is clean and easy to query.

Second Normal Form (2NF):

All non-key attributes fully depend on the primary key. In cases where composite keys are used (like in mapping tables), no attribute depends on just part of the key. This removes partial dependency and avoids unnecessary duplication.

Third Normal Form (3NF):

There are no indirect dependencies between attributes. Every non-key attribute depends only on the primary key and not on another non-key attribute. For example, theatre details are stored separately rather than being repeated in multiple places.

Update During Development :

1. Allow multiple users per booking :

A separate table is introduced to connect bookings with multiple users instead of assuming one booking belongs to only one user.

Booking_Users Table

Attributes:

- booking_id
- user_id

Primary Key:

- (booking_id, user_id)

Foreign Keys:

- booking_id → Bookings(booking_id)
- user_id → Users(user_id)

2. Map each seat to a specific user

Since each seat must belong to a particular user, an additional mapping is required.

Seat_User Mapping Table

Attributes:

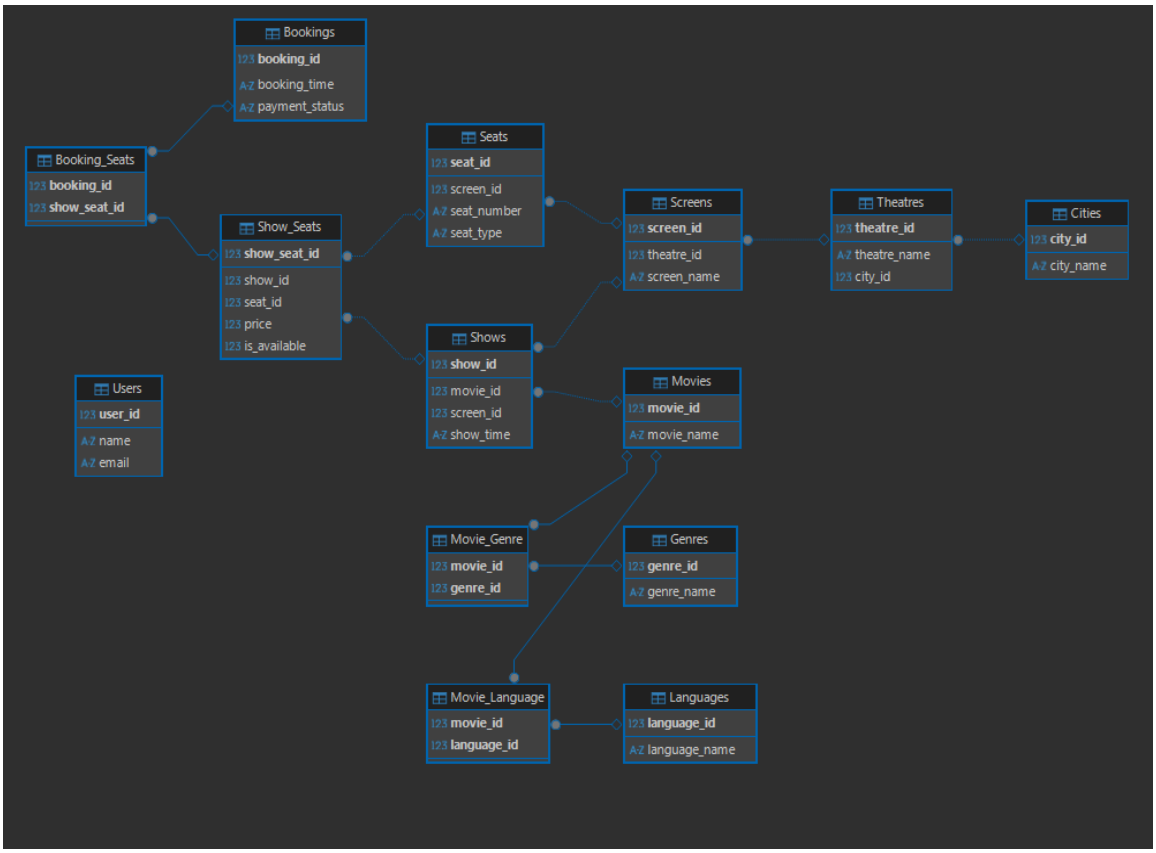
- booking_id
- show_seat_id
- user_id

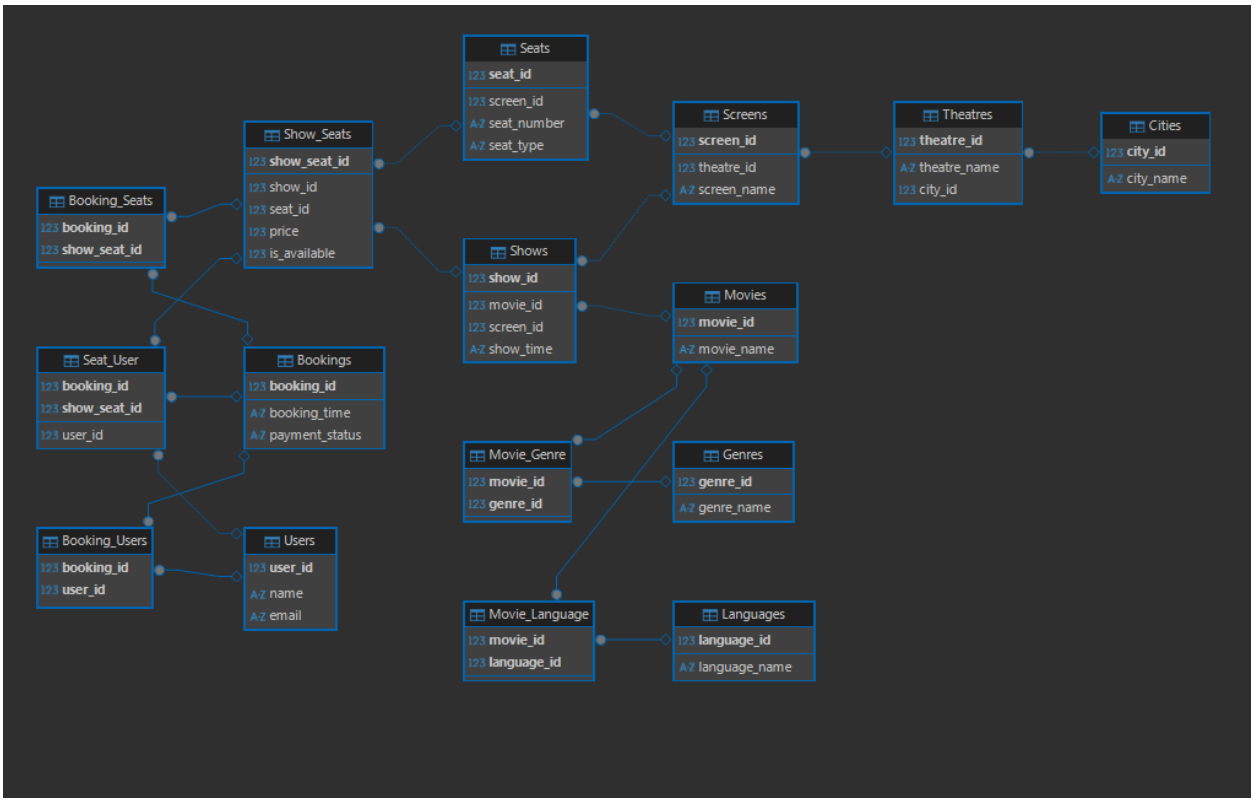
Primary Key:

- (booking_id, show_seat_id)

Foreign Keys:

- booking_id → Bookings(booking_id)
- show_seat_id → Show_Seats(show_seat_id)
- user_id → Users(user_id)


Bookings
booking_id
booking_time
payment_status
Booking_Seats
booking_id
show_seat_id
Seats
seat_id
screen_id
seat_number
seat_type
Show_Seats
show_seat_id
show_id
seat_id
price
is_available
Screens
screen_id
theatre_id
screen_name
Theatres
theatre_id
theatre_name
city_id
Cities
city_id
city_name
Shows
show_id
movie_id
screen_id
show_time
Users
user_id
name
email
Movies
movie_id
movie_name
Movie_Genre
movie_id
genre_id
Genres
genre_id
genre_name
Movie_Language
movie_id
language_id
Languages
language_id
language_name


Seats
seat_id
screen_id
seat_number
seat_type
Screens
screen_id
theatre_id
screen_name
Theatres
theatre_id
theatre_name
city_id
Cities
city_id
city_name
Show_Seats
show_seat_id
show_id
seat_id
price
is_available
Booking_Seats
booking_id
show_seat_id
Shows
show_id
movie_id
screen_id
show_time
Movies
movie_id
movie_name
Seat_User
booking_id
show_seat_id
user_id
Bookings
booking_id
booking_time
payment_status
Movie_Genre
movie_id
genre_id
Genres
genre_id
genre_name
Booking_Users
booking_id
user_id
Users
user_id
name
email
Movie_Language
movie_id
language_id
Languages
language_id
language_name

SECTION C — QUERIES AND TRANSACTIONS

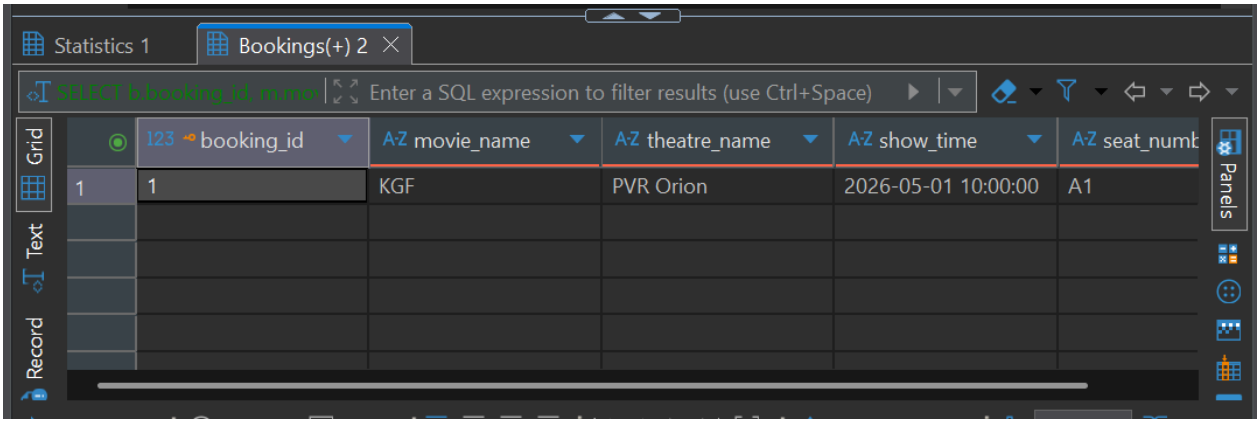
SQL Queries:

1. Get all bookings of a user within a date range :

This query shows all bookings made by a particular user between two dates. It also displays movie name, theatre, show time, and seats booked.

```
SELECT
    b.booking_id,
    m.movie_name,
    t.theatre_name,
    s.show_time,
    se.seat_number
FROM Bookings b
JOIN Booking_Users bu ON b.booking_id = bu.booking_id
```

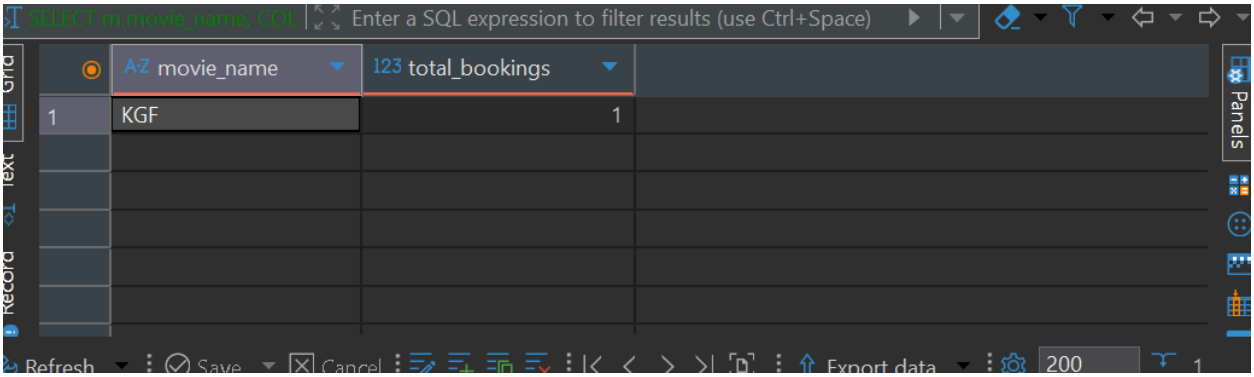
```
JOIN Users u ON bu.user_id = u.user_id
JOIN Booking_Seats bs ON b.booking_id = bs.booking_id
JOIN Show_Seats ss ON bs.show_seat_id = ss.show_seat_id
JOIN Seats se ON ss.seat_id = se.seat_id
JOIN Shows s ON ss.show_id = s.show_id
JOIN Movies m ON s.movie_id = m.movie_id
JOIN Screens sc ON s.screen_id = sc.screen_id
JOIN Theatres t ON sc.theatre_id = t.theatre_id
WHERE u.user_id = 1
AND b.booking_time BETWEEN '2026-01-01' AND '2026-12-31';
```



2. Find the most frequently booked movie :

This query finds which movie has the highest number of bookings.

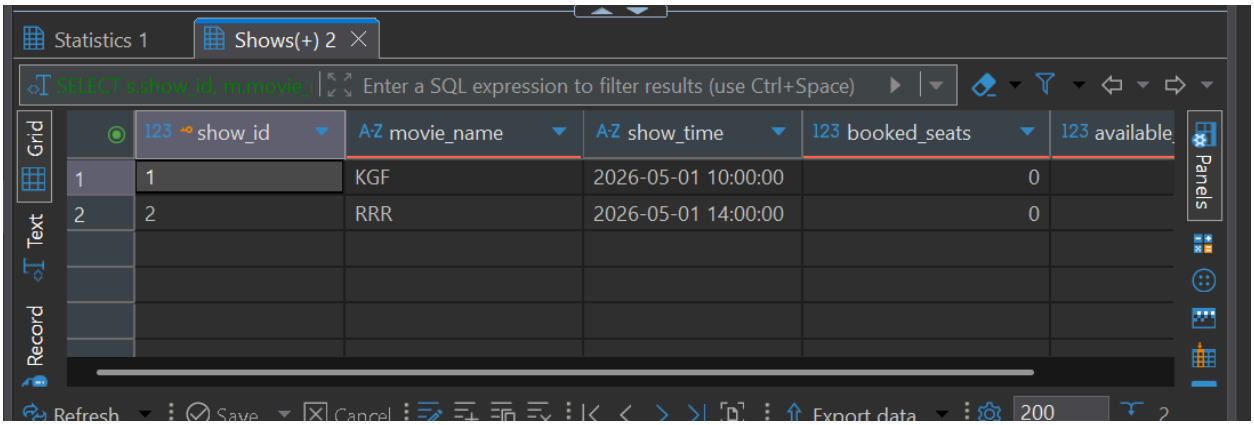
```
SELECT
    m.movie_name,
    COUNT(b.booking_id) AS total_bookings
FROM Bookings b
JOIN Booking_Seats bs ON b.booking_id = bs.booking_id
JOIN Show_Seats ss ON bs.show_seat_id = ss.show_seat_id
JOIN Shows s ON ss.show_id = s.show_id
JOIN Movies m ON s.movie_id = m.movie_id
GROUP BY m.movie_name
ORDER BY total_bookings DESC
LIMIT 1;
```



3. Shows in a theatre with booked and available seats :

This query lists all shows in a theatre on a given date and shows how many seats are booked and how many are still available.

```
SELECT
    s.show_id,
    m.movie_name,
    s.show_time,
    COUNT(CASE WHEN ss.is_available = 0 THEN 1 END) AS booked_seats,
    COUNT(CASE WHEN ss.is_available = 1 THEN 1 END) AS available_seats
FROM Shows s
JOIN Movies m ON s.movie_id = m.movie_id
JOIN Screens sc ON s.screen_id = sc.screen_id
JOIN Theatres t ON sc.theatre_id = t.theatre_id
JOIN Show_Seats ss ON s.show_id = ss.show_id
WHERE t.theatre_id = 1
AND DATE(s.show_time) = '2026-05-01'
GROUP BY s.show_id, m.movie_name, s.show_time;
```



4. Transaction for booking seats:

This transaction handles the full booking process safely.

START TRANSACTION;

BEGIN TRANSACTION;

```
SELECT is_available
FROM Show_Seats
WHERE show_seat_id IN (101, 102);

INSERT INTO Bookings (booking_id, booking_time, payment_status)
VALUES (2, CURRENT_TIMESTAMP, 'PENDING');

INSERT INTO Booking_Seats (booking_id, show_seat_id)
VALUES (2, 101), (2, 102);

UPDATE Show_Seats
SET is_available = 0
WHERE show_seat_id IN (101, 102)
AND is_available = 1;
```

COMMIT;

If any seat is already booked, the transaction should be rolled back manually:

ROLLBACK;

Bookings 1 | Statistics 2

select * from Bookings | Enter a SQL expression to filter results (use Ctrl+Space)

	booking_id	booking_time	payment_status
1	1	2026-05-01 09:00:00	PAID
2	2	2026-04-28 16:02:02	PENDING

Booking_Seats 1 | Statistics 2

select * from Booking_Seats | Enter a SQL expression to filter results (use Ctrl+Space)

	booking_id	show_seat_id
1	1	1
2	1	2
3	1	101
4	1	102
5	2	101
6	2	102

Column: payment_status VARCHAR